

→ For each loop :-

The for each loop is used to **Iterate (traversal)** through all element of an array or collection without using an index.

→ Why do we need for each loop?

let,

```
int[] arr = {10, 20, 30, 40, 50};
```

Using a normal for loop:

```
for (int i = 0; i < arr.length; i++) {  
    cout(arr[i]);  
}
```

here, we declare:

- i
- check condition
- increment i
- write arr[i]

With for-each loop:

```
for (int ele : arr) {  
    cout(ele);  
}
```

Note:- for each loop is not use in taking T/P in array because it is a temporary variable which copy element of an array not the index of that array.

MON TUE WED THU FRI SAT SUN

Page No. 1

Date: _____

index of

Syntax:-

```
for(dataType variable : arrayofCollection)
{
    //statements
}
```

Eg;

```
for(int ele : arr)
```

- int → data type of array element
- ele → Temporary variable
- : → Means "from" or "in"
- arr → Array being traversed

We use it as:-

for each element in array, stored it in ele and execute the loop body

Eg;

```
import java.util.Scanner;
class pogo {
    psum {
        int[] arr = {10, 20, 30, 40, 50};
        for(int ele : arr) {
            sum(ele);
        }
    }
}
```

→ Shallow Copy :- & Deep Copy :-
 When we copy an array in java, there are two ways:
 1. Shallow copy (Reference copy)
 2. Deep copy (Actual copy)

As we know,
 when we create an array:

```
int[] arr = { 10, 20, 30 };
```

the variable `arr` does not store the array itself, it stores the address (reference) of the array object.

<code>arr[0]</code>	<code>arr[1]</code>	<code>arr[2]</code>
10	20	30

→ Shallow Copy :-
 A shallow copy means copying only the reference (address) of the array, not the actual element.

i.e. both variable point to the same array.

Syntax:

```
int[] arr2 = arr1; // shallow copy
```

(No new array is created)

```

Eg; class pogo {
    psvm {
        int[] arr = {10, 20, 30, 40};
        int[] arr1 = arr; // copying in arr1 from arr
        for(int ele : arr1)
            sout(ele);
    }
}

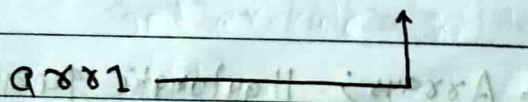
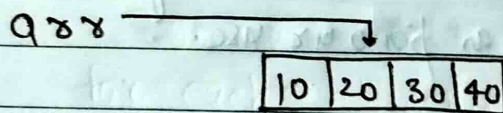
```

// now updating arr1 which reflects in arr:

```

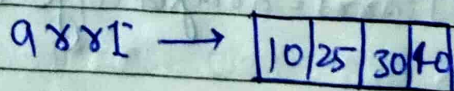
arr1[1] = 25; // updating arr1
for(int ele : arr) // printing arr
    sout(ele);
}
}

```



Now updating

arr1[1] = 25



but arr →

10	25	30	40
----	----	----	----

referencing to same array so arr[1] also change.

→ Deep copy :-

A deep copy means creating a new array and copying all elements into it.

here, now each variable points to a different array.

two common ways

① Using Arrays.copyOf()

Syntax:-

```
int arr2[] = Arrays.copyOf(arr1, arr1.length);
```

arr1.length → here we can change the size of index as per our use.

Eg;

```
import java.util.*;
// automatic generate when use Arrays.copyOf()
```

```
class Pogo {
```

```
    psum {
```

```
        int[] arr = {10, 20, 30, 40};
```

```
        int[] arr1 = Arrays.copyOf(arr, arr.length);
```

```
        // or Use int[] arr1 = Arrays.copyOf(arr, 10)
```

```
        for (int ele : arr1)
```

```
            Sout(ele);
```

int arr[1] = 25; // updating arr

```
for (int ele : arr)
    cout << ele;
}
```

arr →

10	20	30	40
----	----	----	----

arr1 →

10	20	30	40
----	----	----	----

arr[1] = 25

arr →

10	25	30	40
----	----	----	----

arr1 →

10	20	30	40
----	----	----	----

② Using for loop as we do in C/C++ lang.

int[] arr = {10, 20, 30};

int[] arr1 = new int[3];

for (int i = 0; i < 3; i++)

arr1[i] = arr[i];

for (int ele : arr1)

cout << ele;



ArrayList :-

ArrayList is a class in the **java collection framework** (java.util package) that is used to store a **dynamic (resizable)** array of elements.

Unlike normal arrays, the size of an **ArrayList** can **grow or shrink automatically**.

→ Why do we need ArrayList?

Problem with Normal array

```
int[] arr = new new int[5];
```

This array can store only **5** elements.

If we try to store 6th element:

```
arr[5] = 60;
```

it gives an error:

ArrayIndexOutOfBoundsException

Solution:-

ArrayList

Syntax:-

```
ArrayList<Integer> arr = new ArrayList<> ( );
```

↑
size

→ Creating an ArrayList :-

```
import java.util.ArrayList;
```

```
class pogo {
```

```
    psvm {
```

```
        ArrayList<Integer> arr = new ArrayList<>(1);
```

```
        sout(arr);
```

Output []

→ Adding element :-

```
add(  ); or add(index, value);
```

↑
value

```
→ ArrayList<Integer> arr = new ArrayList<>(1);
```

```
arr.add(20); // (0, 20)
```

```
arr.add(30); // (1, 30)
```

```
arr.add(50); // (2, 50)
```

```
sout(arr);
```

Output [20, 30, 50]

- > Accessing element :-

`get(index)`

`cout(a[0].get(1));`

Output - 30.

→ Updating elements :-

`set(index, value)`

~~`cout(a[0].set(1, 100));`~~

`a[0].set(1, 100);`

`cout(a[0]);`

Output: [20, 100, 50]

→ Remove an element

`remove(index)`

`a[0].remove(1);`

`cout(a[0]);`

Output → [20, 50]

(means two different array element.)

Q. Find the doublet in the array whose sum is equal to the given value x .

⇒

arr = { 2, 5, 7, -1 } , let $x = 9$
 $\underbrace{2 + 7}_{= 9} = 9 \checkmark$

```

import java.util.Scanner;
class pogo {
    psum {
        Scanner sc = new Scanner(System.in);
        sout sout("Enter size of an array");
        int n = sc.nextInt();
        int[] arr = new int[n];
        sout("Enter data");
        for(int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        sout("Enter a given value (x)");
        int x = sc.nextInt();

        for(int i = 0; i < n; i++) {
            for(int j = i + 1; j < n; j++) {
                if (arr[i] + arr[j] == x)
                    sout(arr[i] + " + " + arr[j] + " = " + x);
            }
        }
    }
}
    
```

→ 2-D Array :-

A 2-Dimensional Array (2-D Array) is an array that stores data in form of **rows** & **columns** (like a matrix or table).

It is also called a **Matrix**.

Syntax :-

```
int[][] arr = new int[3][4]
```

meaning :

- 3 rows
- 4 columns

Row	0	1	2	3
0	arr[0][0]	arr[0][1]	arr[0][2]	arr[0][3]
1	arr[1][0]	arr[1][1]	arr[1][2]	arr[1][3]
2	arr[2][0]	arr[2][1]	arr[2][2]	arr[2][3]

→ Creating & initializing :

```
int[][] arr = { {10, 20, 30}, {40, 50, 60}, {70, 80, 90} }
```

	0	1	2
0	10	20	30
1	40	50	60
2	70	80	90

→ Access element :-

`arr[row][column];`

→ Java program to access element :-

```
class pogo {
```

```
    psum {
```

```
        int[][] arr = { {1,2,3}, {4,5,6}, {7,8,9} };
```

```
        System.out.println(arr[0][2]); // 3
```

```
        System.out.println(arr[1][1]); // 5
```

```
        System.out.println(arr[2][0]); // 7
```

```
    }
```

```
}
```

→ java program of 2-D array taking it from user

```
import java.util.Scanner;
```

```
class pogo {
```

```
    psum {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        System.out.println("enter size of a row");
```

```
        int x = sc.nextInt();
```

```

        Sout("enter size of a column");
        int y = sc.nextInt();
    
```

```

        int[][] arr = new int[x][y];
    
```

```

        Sout("enter data");
    
```

```

        for(int i=0; i<x; i++){
            for(int j=0; j<y; j++){
                arr[i][j] = sc.nextInt();
            }
        }
    
```

```

        sout("output");
    
```

```

        for(int i=0; i<x; i++){
            for(int j=0; j<y; j++){
                Sout(arr[i][j] + " ");
            }
        }
    
```

```

        Sout("\n");
    }
}

```

→ String :-

A string is a sequence (collection) of character enclosed in double quotes (" ").

→ To print without using String class :-

```
class pogo {
    public static void main(String[] args) {
        char[] ch = {'A', 'K', 'A', 'S', 'H'};
        for (int i = 0; i < ch.length; i++)
            sout (ch[i]);
    }
}
```

→ Using String class :-

```
class pogo {
    public static void main (String[] args) {
        String st = "AKASH";
        sout (st);
    }
}
```

→ Taking String input :-

```
import java.util.Scanner;
```

```
class Pogo {
```

```
    psum {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        System.out.println("Enter your name");
```

```
        String st = sc.next();
```

```
        System.out.println(st);
```

```
    }
```

```
}
```

Here is one problem

Output console :-

Enter your name

Akash Kumar ↵ (enter)

Akash

To print we use

```
    nextLine();
```

```
import java.util.*; Scanner;  
class pogo {
```

```
    psum {
```

```
        Scanner sc = new Scanner (System.in);
```

```
        Sout ("Enter the name");
```

```
        String @ = sc.nextLine();
```

```
        Sout (@);
```

```
    }
```

```
}
```

• next() :-

Reads only one word.

• nextLine() :-

Reads the entire line.

Akash